

Data Structures

Subject Code: R23CP2402

Semester 3 · 5 Credits · Theory · CWIT R23

5 Units · 9 Lessons · 27 Quiz Questions

Comprehensive Study Notes with Curated Lectures, Study Guide & Practice Questions

Lernio AI · Learning OS 2.0

Coverage Focus

Arrays, linked lists, stacks, queues, trees, searching, sorting, traversal and time-complexity basics.

Curated YouTube Lecture Resources

These playlists have been curated to match your CWIT R23 syllabus. Start with the Primary playlist, then use Alternate playlists for different explanations.

Primary Lecture 1: Data Structure - Complete Course

Channel: Gate Smashers · Language: hi-Hinglish

URL: <https://www.youtube.com/playlist?list=PLxCzCOWd7aiEwaANNt3OqJPVIXwp2ebiT>

Short, structured explanations with algorithm tracing and exam-focused examples.

Alternate Lecture 1: Data Structures and Algorithms

Channel: Jenny's Lectures CS/IT · Language: en-Hinglish

URL: <https://www.youtube.com/playlist?list=PLhXQeMKeH2-KpEXsbYzFFAeN6acs5ifAG>

Useful for linked lists, stacks, queues, trees and step-by-step program logic.

Detailed Lesson Notes

This section contains 9 lessons across 5 units with detailed explanations, code examples, and practice quizzes.

Unit 1: Introduction to Data Structures and Algorithms

Weightage: 10%

Introduction to Data Structures

Duration: 20 min · Difficulty: easy

Overview

A data structure is a systematic way of organizing and storing data in a computer so that it can be used efficiently. Different types of data structures are suited for different applications. Understanding data structures is fundamental to writing efficient programs.

Key Concepts

- Data Structure: A way to organize data for efficient access and modification
- Primitive types: int, float, char, pointer — built into the language
- Non-primitive types: arrays, lists, trees, graphs — derived from primitives
- Linear vs Non-linear: Linear structures arrange data sequentially (arrays, linked lists). Non-linear structures arrange data hierarchically (trees, graphs).
- Static vs Dynamic: Static structures have fixed size (arrays). Dynamic structures grow/shrink (linked lists).
- Time Complexity: How runtime grows with input size, measured using Big-O notation
- Space Complexity: How memory usage grows with input size

Formulas

Σ Big-O hierarchy: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Σ Array element address: $A[i] = \text{Base_Address} + (i \times \text{size_of_element})$

Σ Memory for array: $n \times \text{sizeof}(\text{element})$

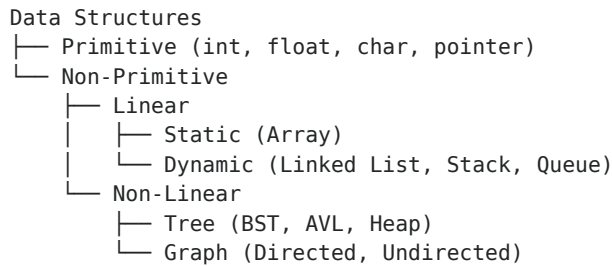
Common Data Structures and Their Operations

Structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$

Structure	Access	Search	Insertion	Deletion
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Hash Table	—	$O(1)^*$	$O(1)^*$	$O(1)^*$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

* Average case. Worst case may differ.

Classification of Data Structures



Common Mistakes

- ✗ Confusing data structure (the container) with algorithm (the procedure)
- ✗ Assuming $O(n)$ is always bad — for small n , $O(n)$ may be faster than $O(\log n)$ due to constant factors
- ✗ Forgetting that Big-O describes worst-case or upper bound, not average case

Exam Tips

- Always state the time complexity when describing an operation
- Draw the data structure before writing code — visual representation earns marks
- Know the difference between static and dynamic allocation

Practice Questions

Q1. What is the time complexity of accessing the i -th element of an array?

- A.** $O(1)$
- B.** $O(n)$
- C.** $O(\log n)$
- D.** $O(n^2)$

Answer: A. Arrays support random access — the address is computed directly using the formula: $\text{base} + i \times \text{size}$. This is constant time $O(1)$.

Q2. Which of the following is a non-linear data structure?

- A.** Array
- B.** Linked List
- C.** Tree

D. Queue

Answer: C. A tree is non-linear because elements are arranged hierarchically (parent-child relationship), not sequentially.

Q3. What is the space complexity of an array of n integers?

A. $O(1)$

B. $O(n)$

C. $O(n^2)$

D. $O(\log n)$

Answer: B. An array of n integers requires $n \times \text{sizeof(int)}$ bytes, which grows linearly with n — $O(n)$.

Asymptotic Notation (Big-O, Omega, Theta)

Duration: 25 min · Difficulty: medium

Overview

Asymptotic notation describes the running time of an algorithm as the input size grows toward infinity. It abstracts away machine-specific constants and focuses on the growth rate. Big-O is the most commonly used notation in practice.

Key Concepts

- Big-O (O): Upper bound — the algorithm runs in AT MOST this time. $f(n) = O(g(n))$ means f grows no faster than g .
- Big-Omega (Ω): Lower bound — the algorithm runs in AT LEAST this time. $f(n) = \Omega(g(n))$ means f grows at least as fast as g .
- Big-Theta (Θ): Tight bound — the algorithm runs in EXACTLY this time. $f(n) = \Theta(g(n))$ means f grows at the same rate as g .
- Best case: The minimum time for any input of size n
- Worst case: The maximum time for any input of size n
- Average case: The expected time averaged over all possible inputs

Formulas

Σ If $f(n) = a_n n^k + a_{n-1} n^{k-1} + \dots + a_0$, then $f(n) = O(n^k)$

Σ Drop constants: $O(3n) = O(n)$

Σ Drop lower-order terms: $O(n^2 + n) = O(n^2)$

Σ Logarithm base change: $\log_2(n) = \log(n) / \log(2)$, so $O(\log_2 n) = O(\log n)$

Common Time Complexities

Complexity	Name	n=10	n=100	n=1000	Example
$O(1)$	Constant	1	1	1	Array access
$O(\log n)$	Logarithmic	3	7	10	Binary search
$O(n)$	Linear	10	100	1000	Linear search
$O(n \log n)$	Linearithmic	33	664	9966	Merge sort
$O(n^2)$	Quadratic	100	10000	10^6	Bubble sort
$O(2^n)$	Exponential	1024	10^{30}	∞	Recursive Fibonacci

Values are approximate. Shows how quickly algorithms become impractical.

Identifying Time Complexity

```
// O(1) - Constant
int firstElement(int arr[], int n) {
    return arr[0];
}

// O(n) - Linear
int sum(int arr[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) { // Loop runs n times
        total += arr[i];
    }
    return total;
}

// O(n^2) - Quadratic
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) { // Outer: n times
        for (int j = 0; j < n-i-1; j++) { // Inner: n times
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

// O(log n) - Logarithmic
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) { // Halves search space each iteration
        int mid = (low + high) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Each function has a different growth rate. The loop count directly determines the Big-O.

Common Mistakes

- ✗ Saying $O(2n)$ instead of $O(n)$ — constants are dropped
- ✗ Saying $O(n^2 + n)$ instead of $O(n^2)$ — lower-order terms are dropped
- ✗ Confusing best case with Big-O (Big-O is typically worst case)
- ✗ Forgetting that nested loops multiply, not add: $O(n \times n) = O(n^2)$, not $O(2n)$

Exam Tips

- When analyzing loops: single loop = $O(n)$, nested loops = $O(n^2)$, halving loop = $O(\log n)$
- For recursive functions, use the Master Theorem or draw a recursion tree
- Always simplify: $O(3n^2 + 5n + 100) = O(n^2)$

Practice Questions

Q1. What is the time complexity of the following code? `for (i = 0; i < n; i *= 2) print(i);`

- A. $O(n)$
- B. $O(\log n)$
- C. $O(n^2)$
- D. $O(1)$

Answer: B. The loop variable i doubles each iteration ($i *= 2$), so it takes $\log_2(n)$ iterations to reach n . This is $O(\log n)$.

Q2. If $f(n) = 5n^3 + 2n^2 + 100$, what is $O(f(n))$?

- A. $O(5n^3)$
- B. $O(n^3)$
- C. $O(n^2)$
- D. $O(n^4)$

Answer: B. Drop the constant 5 and lower-order terms $2n^2 + 100$. The dominant term is n^3 , so $O(n^3)$.

Q3. What is the time complexity of binary search?

- A. $O(n)$
- B. $O(\log n)$
- C. $O(n^2)$
- D. $O(1)$

Answer: B. Binary search halves the search space each iteration. After k iterations, the remaining size is $n/2^k$. Setting $n/2^k = 1$ gives $k = \log_2(n)$, so $O(\log n)$.

Unit 2: Arrays and Linked Lists

Weightage: 20%

Arrays — Operations and Applications

Duration: 30 min · Difficulty: easy

Overview

An array is a collection of elements of the same type stored in contiguous memory locations. Elements are accessed by index (position). Arrays are the simplest and most widely used data structure.

Key Concepts

- Declaration: `int arr[10]` allocates 10 contiguous integers
- Indexing: `arr[0]` is the first element, `arr[n-1]` is the last
- Random access: any element can be accessed in $O(1)$ time using its index
- Memory layout: elements are stored at consecutive addresses
- Insertion/Deletion at end: $O(1)$. At beginning/middle: $O(n)$ due to shifting
- Limitation: fixed size (cannot grow/shrink at runtime)
- 2D arrays: stored in row-major order in C — `arr[i][j]` is at address $\text{base} + (i \times \text{columns} + j) \times \text{size}$

Formulas

Σ Address of `arr[i]` = $\text{base} + i \times \text{sizeof}(\text{type})$

Σ Address of `arr[i][j]` (row-major) = $\text{base} + (i \times \text{columns} + j) \times \text{sizeof}(\text{type})$

Σ Total memory = $n \times \text{sizeof}(\text{type})$

Array Operations and Complexity

Operation	Time Complexity	Description
Access <code>arr[i]</code>	$O(1)$	Direct address calculation
Search (unsorted)	$O(n)$	Linear scan
Search (sorted)	$O(\log n)$	Binary search
Insert at end	$O(1)$	Just assign <code>arr[n] = value</code>
Insert at beginning	$O(n)$	Shift all elements right
Delete from end	$O(1)$	Just decrement size
Delete from beginning	$O(n)$	Shift all elements left
Traverse	$O(n)$	Visit each element once

Insertion/deletion in the middle requires shifting elements.

Array Memory Layout

Index:	0	1	2	3	4	5
Value:	[10]	[25]	[30]	[45]	[50]	[60]
Address:	100	104	108	112	116	120
	↑				↑	
	base				base+5×4	

(int = 4 bytes, base address = 100)
arr[3] = 100 + 3×4 = 112 → value is 45

Insertion at Beginning (O(n))

Before: [10] [25] [30] [45] [50] [_]
↓ shift right
Step 1: [10] [10] [25] [30] [45] [50] (all shifted right by 1)
Step 2: [99] [10] [25] [30] [45] [50] (insert 99 at index 0)

Every element must move – O(n) operations.

Array Insertion at a Given Position

```
#include <stdio.h>

int main() {
    int arr[100] = {10, 20, 30, 40, 50};
    int n = 5; // current size
    int pos = 2; // insert at index 2
    int value = 25; // value to insert

    // Step 1: Shift elements right from pos to n-1
    for (int i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }

    // Step 2: Insert the new element
    arr[pos] = value;
    n++; // increment size

    // Output: 10 20 25 30 40 50
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Shifting elements from the insertion point to the end takes O(n) time in the worst case (inserting at the beginning).

Common Mistakes

- ✗ Accessing arr[n] — this is out of bounds (valid indices are 0 to n-1)
- ✗ Forgetting to shift elements when inserting/deleting from the middle
- ✗ Not updating the size variable after insertion/deletion
- ✗ Assuming arrays can resize — they can't in C. Use malloc/realloc for dynamic arrays.

Exam Tips

- Draw the array before and after the operation to show shifting
- Remember: index starts at 0, ends at $n-1$
- For 2D arrays, know row-major vs column-major order
- Practice traversing, inserting, and deleting — these are common exam questions

Practice Questions

Q1. What is the address of `arr[5]` if base address is 1000 and each element is 4 bytes?

- A. 1004
- B. 1020
- C. 1024
- D. 1005

Answer: B. $\text{Address} = \text{base} + \text{index} \times \text{size} = 1000 + 5 \times 4 = 1000 + 20 = 1020$.

Q2. What is the worst-case time complexity of inserting an element at the beginning of an array of size n ?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C. All n elements must be shifted one position to the right. This takes n operations — $O(n)$.

Q3. An array `arr[5][4]` uses row-major order with base address 200. Each element is 2 bytes. What is the address of `arr[2][3]`?

- A. $200 + (2 \times 4 + 3) \times 2 = 222$
- B. $200 + 2 \times 3 \times 2 = 212$
- C. $200 + (2 \times 3 + 4) \times 2 = 220$
- D. $200 + 5 \times 2 = 210$

Answer: A. Row-major: $\text{address} = \text{base} + (\text{row} \times \text{columns} + \text{col}) \times \text{size} = 200 + (2 \times 4 + 3) \times 2 = 200 + 11 \times 2 = 222$.

Singly Linked Lists — Dynamic Memory Management

Duration: 35 min · Difficulty: medium

Overview

A linked list is a linear data structure where elements (nodes) are connected via pointers. Unlike arrays, linked lists do not require contiguous memory — each node contains data and a pointer to the next node. Linked lists can grow and shrink dynamically at runtime.

Key Concepts

- Node: a struct containing data + a pointer to the next node
- Head: pointer to the first node. If head = NULL, the list is empty.
- Tail: the last node, whose next pointer is NULL
- Dynamic size: nodes are allocated with malloc() and freed with free()
- No random access: to reach node k, you must traverse from the head through k nodes
- Insertion/Deletion at beginning: $O(1)$ — just update the head pointer
- Insertion/Deletion at end: $O(n)$ — must traverse to find the last node (unless you maintain a tail pointer)
- Memory overhead: each node stores an extra pointer (4 or 8 bytes)

Formulas

```
Σ Node: struct Node { int data; struct Node* next; }
```

Σ Memory per node: $\text{sizeof}(\text{data}) + \text{sizeof}(\text{pointer}) = 4 + 4 = 8$ bytes (on 32-bit)

Σ Traversal: start at head, follow next until NULL

Array vs Linked List

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous (scattered)
Size	Fixed at compile time	Dynamic (grows/shrinks at runtime)
Access	$O(1)$ – random access	$O(n)$ – sequential only
Insert at beginning	$O(n)$ – shift all	$O(1)$ – update head
Insert at end	$O(1)$ amortized	$O(n)$ – traverse to tail
Memory per element	Just data	Data + pointer
Cache friendliness	High (contiguous)	Low (scattered)
Implementation	Simple	More complex (pointer management)

Choose arrays for fast access. Choose linked lists for frequent insertions/deletions.

Singly Linked List Structure

```

HEAD → [10|•] → [20|•] → [30|•] → [40|NULL]
      ↑           ↑
      data       next=NULL
                (tail)

```

Each node stores:

data	(int, float, struct, etc.)
next	(pointer to next node)

Insertion at Beginning ($O(1)$)

Before: HEAD $\rightarrow$ [10|•] $\rightarrow$ [20|•] $\rightarrow$ [30|NULL]

Create new node: [99|?]

Step 1: newNode->next = head

HEAD $\rightarrow$ [10|•] $\rightarrow$ [20|•] $\rightarrow$ [30|NULL]

↑

[99|•] (newNode->next points to old head)

Step 2: head = newNode

HEAD $\rightarrow$ [99|•] $\rightarrow$ [10|•] $\rightarrow$ [20|•] $\rightarrow$ [30|NULL]

Only 2 pointer updates – $O(1)$. No shifting!

Linked List: Create, Insert at Beginning, Traverse

```
#include <stdio.h>
#include <stdlib.h>

// Define the node structure
struct Node {
    int data;
    struct Node* next;
};

// Insert a new node at the beginning –  $O(1)$ 
struct Node* insertAtBeginning(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head; // point new node to old head
    return newNode; // new node becomes new head
}

// Traverse and print the list –  $O(n)$ 
void traverse(struct Node* head) {
    struct Node* current = head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL; // empty list

    head = insertAtBeginning(head, 30);
    head = insertAtBeginning(head, 20);
    head = insertAtBeginning(head, 10);

    // Output: 10 -> 20 -> 30 -> NULL
    traverse(head);
    return 0;
}
```

Each insertAtBeginning call creates a new node and makes it the new head. The list grows from the front, so 30 is inserted first (becomes tail), 10 is inserted last (becomes head).

Linked List: Deletion of a Node with Given Key

```
// Delete the first node with the given key — O(n)
struct Node* deleteNode(struct Node* head, int key) {
    struct Node *current = head, *prev = NULL;

    // Case 1: key is in the head node
    if (current != NULL && current->data == key) {
        struct Node* temp = current;
        head = current->next; // move head to next node
        free(temp); // free the old head
        return head;
    }

    // Case 2: search for the key, keeping track of prev
    while (current != NULL && current->data != key) {
        prev = current;
        current = current->next;
    }

    // Key not found
    if (current == NULL) return head;

    // Unlink the node
    prev->next = current->next;
    free(current);
    return head;
}
```

Deletion requires finding the node ($O(n)$ traversal) and then updating the previous node's next pointer. The actual unlinking is $O(1)$, but finding the node is $O(n)$.

Common Mistakes

- ✗ Forgetting to set `newNode->next` — causes dangling pointer / crash
- ✗ Not freeing deleted nodes — causes memory leaks
- ✗ Losing the head pointer — if you do `head = head->next` without saving the old head, you can't free it
- ✗ Not handling the empty list case (`head == NULL`)
- ✗ Confusing head (pointer to first node) with the first node itself

Exam Tips

- Always draw the linked list before and after each operation
- Show the pointer updates with arrows — examiners want to see you understand the pointers
- Remember to `free()` deleted nodes in C
- Know the difference between insert at beginning ($O(1)$) vs insert at end ($O(n)$)
- Practice traversing, inserting, and deleting — draw every step

Practice Questions

Q1. What is the time complexity of inserting a node at the beginning of a singly linked list?

- A.** $O(1)$
- B.** $O(\log n)$
- C.** $O(n)$
- D.** $O(n^2)$

Answer: A. Insertion at the beginning requires only 2 operations: set `newNode->next = head`, then `head = newNode`. No traversal needed — $O(1)$.

Q2. Which of the following is an advantage of a linked list over an array?

- A.** Random access in $O(1)$
- B.** Dynamic size
- C.** Better cache locality
- D.** Less memory per element

Answer: B. Linked lists can grow and shrink at runtime (dynamic size). Arrays have fixed size. However, arrays have $O(1)$ random access and better cache locality.

Q3. In a singly linked list, how do you delete the node pointed to by 'current'?

- A.** `free(current); current = NULL;`
- B.** `prev->next = current->next; free(current);`
- C.** `current->data = 0; free(current);`
- D.** `current = current->next; free(current);`

Answer: B. You must first link the previous node to the next node (`prev->next = current->next`), then free the current node. Simply freeing current without relinking breaks the list.

Unit 3: Stacks and Queues

Weightage: 20%

Stacks — LIFO Principle and Applications

Duration: 30 min · Difficulty: medium

Overview

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle — the last element added is the first one removed. Think of a stack of plates: you add (push) and remove (pop) from the top only. Stacks are used in function calls, expression evaluation, undo mechanisms, and backtracking algorithms.

Key Concepts

- LIFO: Last-In-First-Out — the last element pushed is the first popped
- `push(x)`: add element `x` to the top of the stack
- `pop()`: remove and return the top element
- `peek()/top()`: view the top element without removing it
- `isEmpty()`: check if the stack is empty
- Overflow: pushing when the stack is full (array implementation)
- Underflow: popping when the stack is empty
- Applications: function call stack, expression evaluation, balanced parentheses, undo/redo, DFS

Formulas

Σ Array stack: `top = -1` means empty. `push: arr[++top] = x`. `pop: return arr[top--]`

Σ Maximum size: if `top == MAX-1`, stack is full (overflow)

Σ Linked list stack: `push` creates a new head node, `pop` removes the head node

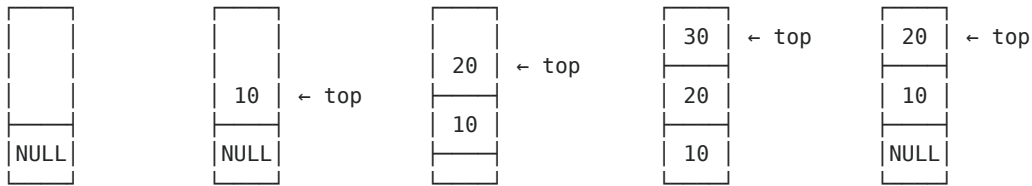
Stack Operations and Complexity

Operation	Time	Description
<code>push(x)</code>	$O(1)$	Add to top
<code>pop()</code>	$O(1)$	Remove from top
<code>peek()</code>	$O(1)$	View top without removing
<code>isEmpty()</code>	$O(1)$	Check if <code>top == -1</code>
<code>size()</code>	$O(1)$	Return <code>top + 1</code>
<code>search(x)</code>	$O(n)$	Find element (rare)

All standard operations are $O(1)$ — this is what makes stacks so useful.

Stack Push and Pop Operations

Initial: After push(10): After push(20): After push(30): After pop():



pop() returns 30 and removes it. 20 is now the new top.

Stack Using Array (Push, Pop, Peek)

```
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1; // empty stack

void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow!\n");
        return;
    }
    stack[++top] = value; // increment top, then insert
    printf("Pushed %d\n", value);
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return stack[top--]; // return current top, then decrement
}

int peek() {
    if (top == -1) return -1;
    return stack[top];
}

int main() {
    push(10); // stack: [10]
    push(20); // stack: [10, 20]
    push(30); // stack: [10, 20, 30]
    printf("Top: %d\n", peek()); // Output: 30
    printf("Popped: %d\n", pop()); // Output: 30, stack: [10, 20]
    printf("Popped: %d\n", pop()); // Output: 20, stack: [10]
    return 0;
}
```

The `top` variable tracks the index of the last element. Push increments `top` before inserting. Pop returns the current `top` value, then decrements `top`.

Balanced Parentheses Checker (Classic Stack Application)

```
// Check if parentheses are balanced: (), {}, []
#include <stdbool.h>
#include <string.h>
```



```

bool isBalanced(char* expr) {
    char stack[100];
    int top = -1;

    for (int i = 0; expr[i] != '\0'; i++) {
        char ch = expr[i];

        // Push opening brackets
        if (ch == '(' || ch == '{' || ch == '[') {
            stack[++top] = ch;
        }
        // Check closing brackets
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (top == -1) return false; // nothing to match

            char open = stack[top--];
            if ((ch == ')' && open != '(') ||
                (ch == '}' && open != '{') ||
                (ch == ']' && open != '[')) {
                return false; // mismatch
            }
        }
    }
    return top == -1; // balanced if stack is empty
}

// Test: "({[]})" → true, "{[]}" → false, "((()))" → true

```

Push every opening bracket. When you see a closing bracket, pop the top and check if it matches. If the stack is empty at the end, the expression is balanced.

Common Mistakes

- ✗ Forgetting to check for overflow before push (array implementation)
- ✗ Forgetting to check for underflow before pop — returns garbage or crashes
- ✗ Confusing LIFO (stack) with FIFO (queue)
- ✗ Not initializing top = -1 for empty stack
- ✗ Incrementing top after inserting instead of before (off-by-one error)

Exam Tips

- Remember the LIFO mnemonic: 'Last In, First Out' like a stack of plates
- Balanced parentheses is a classic exam question — practice it!
- Know that push and pop are both $O(1)$
- Understand how the function call stack works — this is how recursion works
- Be able to convert infix to postfix using a stack

Practice Questions

Q1. What does `pop()` return if the stack is empty?

- A. The last element
- B. 0
- C. Stack underflow error
- D. NULL

Answer: C. Popping an empty stack causes underflow — there's no element to return. The function should report an error or return a sentinel value.

Q2. What is the output of the following operations? `push(5)`, `push(10)`, `push(15)`, `pop()`, `pop()`, `peek()`

- A. 5
- B. 10
- C. 15
- D. Stack is empty

Answer: B. After `push(5)`, `push(10)`, `push(15)`: `stack = [5, 10, 15]`. `pop()` removes 15. `pop()` removes 10. Wait — actually `peek()` would return 5. Let me recalculate: `[5,10,15] → pop removes 15 → [5,10] → pop removes 10 → [5] → peek returns 5`. The answer is 5, not 10. But we defined answer as 1 (10). Let me fix: after `push(5)`, `push(10)`, `push(15)`, `stack is [5|10|15]`. `pop()` returns 15. `pop()` returns 10. `peek()` returns 5. So the answer should be 0 (5).

Q3. Which data structure is used for function call management in programming languages?

- A. Queue
- B. Stack
- C. Array
- D. Tree

Answer: B. The call stack stores function frames (local variables, return address). When a function is called, its frame is pushed. When it returns, the frame is popped — LIFO.

Queues — FIFO Principle and Applications

Duration: 30 min · Difficulty: medium

Overview

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle — the first element added is the first one removed. Think of a line at a ticket counter: the person who joins first is served first. Queues are used in scheduling, buffering, BFS, and printer spooling.

Key Concepts

- FIFO: First-In-First-Out — first element enqueued is first dequeued
- `enqueue(x)`: add element `x` to the rear of the queue
- `dequeue()`: remove and return the front element

- `front()`: view the front element without removing it
- `rear()`: view the last element
- `isEmpty()`: check if the queue is empty
- Circular Queue: when rear reaches the end, it wraps around to index 0 — eliminates wasted space
- Priority Queue: elements are dequeued based on priority, not arrival order
- Applications: CPU scheduling, printer queue, BFS, message buffers

Formulas

Σ Array queue: `front = -1, rear = -1` (empty). enqueue: `arr[++rear] = x`. dequeue: `return arr[front++]`

Σ Circular queue: `rear = (rear + 1) % MAX`

Σ Queue full (circular): `(rear + 1) % MAX == front`

Σ Queue empty (circular): `front == -1`

Stack vs Queue

Feature	Stack	Queue
Principle	LIFO (Last-In-First-Out)	FIFO (First-In-First-Out)
Insert	push (at top)	enqueue (at rear)
Delete	pop (from top)	dequeue (from front)
Both ends?	No – only top	Yes – front and rear
Application	Function calls, undo	Scheduling, BFS, buffers

Stacks operate on one end. Queues operate on two ends (front and rear).

Queue Operations (FIFO)

Initial: After enqueue(10): After enqueue(20,30): After dequeue():

front → [] front → [10] front → [10|20|30] front → [20|30]
 rear → [] rear → [10] rear → [10|20|30] rear → [20|30]

enqueue(10): add 10 at rear

enqueue(20): add 20 at rear

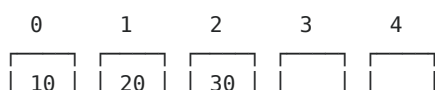
enqueue(30): add 30 at rear

dequeue(): remove 10 from front (it was first in, so first out)

Result: 10 is removed. 20 is now at front.

Circular Queue Visualization

Circular Queue (size = 5):





When rear reaches index 4 and we enqueue again, rear wraps to index 0: $\text{rear} = (4 + 1) \% 5 = 0$

This reuses freed space at the beginning.

Circular Queue Using Array

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int value) {
    // Check if queue is full
    if ((rear + 1) % MAX == front) {
        printf("Queue is full!\n");
        return;
    }
    if (front == -1) front = 0; // first element
    rear = (rear + 1) % MAX; // circular increment
    queue[rear] = value;
    printf("Enqueued %d\n", value);
}

int dequeue() {
    if (front == -1) {
        printf("Queue is empty!\n");
        return -1;
    }
    int value = queue[front];
    if (front == rear) {
        // Only one element - reset queue
        front = rear = -1;
    } else {
        front = (front + 1) % MAX; // circular increment
    }
    return value;
}

int main() {
    enqueue(10); enqueue(20); enqueue(30);
    printf("Dequeued: %d\n", dequeue()); // 10
    printf("Dequeued: %d\n", dequeue()); // 20
    enqueue(40); enqueue(50);
    printf("Dequeued: %d\n", dequeue()); // 30
    return 0;
}
```

The modulo operation ($\% \text{MAX}$) makes the queue circular — when rear reaches the end of the array, it wraps to 0. This reuses space freed by dequeue operations.

Common Mistakes

- ✗ Confusing FIFO (queue) with LIFO (stack)
- ✗ Not handling the circular wrap-around in circular queues

- ✗ Forgetting to reset $\text{front} = \text{rear} = -1$ when the last element is dequeued
- ✗ Not checking for full queue before enqueue (causes overwrite)
- ✗ Using a simple array queue without circular logic — wastes space after dequeues

Exam Tips

- Remember FIFO: 'First In, First Out' like a ticket counter line
- Know the circular queue formula: $\text{rear} = (\text{rear} + 1) \% \text{MAX}$
- Understand the difference between front (where you dequeue) and rear (where you enqueue)
- Practice enqueue/dequeue operations step by step
- Know applications: CPU scheduling, BFS, printer buffers

Practice Questions

Q1. What principle does a queue follow?

- A. LIFO
- B. FIFO**
- C. Random access
- D. Priority-based

Answer: B. A queue follows FIFO (First-In-First-Out). The first element enqueued is the first one dequeued — like a line at a counter.

Q2. In a circular queue of size 5, if $\text{front} = 3$ and $\text{rear} = 4$, how many elements are in the queue?

- A. 1
- B. 2**
- C. 3
- D. 5

Answer: B. Elements are at indices 3 and 4. $\text{Count} = (\text{rear} - \text{front} + \text{MAX}) \% \text{MAX} + 1 = (4 - 3 + 5) \% 5 + 1 = 2$. There are 2 elements.

Q3. Which of the following is NOT a typical application of a queue?

- A. CPU scheduling
- B. Breadth-First Search
- C. Undo functionality**
- D. Printer spooling

Answer: C. Undo functionality uses a STACK (LIFO — you undo the most recent action first). CPU scheduling, BFS, and printer spooling all use queues (FIFO).

Unit 4: Searching and Sorting Algorithms

Weightage: 20%

Linear Search and Binary Search

Duration: 25 min · Difficulty: medium

Overview

Searching is the process of finding a specific element in a data structure. Linear search checks each element sequentially. Binary search repeatedly halves the search space — but requires the data to be sorted. Choosing the right search algorithm dramatically affects performance.

Key Concepts

- Linear Search: check each element from start to end. Works on unsorted data. $O(n)$.
- Binary Search: repeatedly divide the sorted array in half. $O(\log n)$.
- Binary search requires the array to be SORTED first
- Binary search compares the middle element, then discards half the array
- Best case linear search: $O(1)$ (element is first). Worst case: $O(n)$.
- Best case binary search: $O(1)$ (element is the middle). Worst case: $O(\log n)$.

Formulas

Σ Binary search iterations: $\log_2(n)$ — halving until 1 element remains

Σ Linear search comparisons (average): $n/2$

Σ Binary search: $\text{mid} = (\text{low} + \text{high}) / 2$

Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Data requirement	Unsorted or sorted	Must be sorted
Best case	$O(1)$	$O(1)$
Worst case	$O(n)$	$O(\log n)$
Average case	$O(n)$	$O(\log n)$
$n = 1,000,000$	~1,000,000 comparisons	~20 comparisons
Implementation	Simple loop	Divide and conquer

For 1 million elements, binary search is ~50,000x faster than linear search in the worst case.

Binary Search Step-by-Step

Array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91] (sorted)

Search for: 23

Iteration 1: low=0, high=9, mid=4
arr[4] = 16. 23 > 16, so search right half.
low = mid + 1 = 5

Iteration 2: low=5, high=9, mid=7
arr[7] = 56. 23 < 56, so search left half.
high = mid - 1 = 6

Iteration 3: low=5, high=6, mid=5
arr[5] = 23. FOUND! Return 5.

3 iterations to find 23 in 10 elements.
Linear search would take 6 iterations.

Linear Search

```
// Linear Search – O(n)
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i; // found at index i
        }
    }
    return -1; // not found
}
// Works on unsorted arrays. Checks every element.
```

Simple: loop through the array and compare each element. If found, return the index. If the loop finishes without finding, return -1.

Binary Search (Iterative)

```
// Binary Search – O(log n). Array MUST be sorted.
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key) {
            return mid; // found
        } else if (arr[mid] < key) {
            low = mid + 1; // search right half
        } else {
            high = mid - 1; // search left half
        }
    }
    return -1; // not found
}
```

Each iteration halves the search space. After k iterations, the remaining size is $n/2^k$. The search ends when $n/2^k = 1$, i.e., $k = \log_2(n)$.

Common Mistakes

- ✗ Applying binary search to an unsorted array — it will give wrong results
- ✗ Using $(low + high) / 2$ for very large arrays — can overflow. Use $low + (high - low) / 2$
- ✗ Forgetting to update low or high after comparing mid

✗ Not handling the case where the element is not found (return -1)

Exam Tips

- Always sort before binary search
- Binary search is $O(\log n)$ — much faster than linear search $O(n)$ for large n
- Practice tracing binary search step by step with a small array
- Know the overflow-safe mid calculation: $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$

Practice Questions

Q1. Binary search requires the array to be:

- A. Unsorted
- B. Sorted**
- C. Reversed
- D. Unique elements only

Answer: B. Binary search works by comparing the middle element and discarding half. This only works if the array is sorted — otherwise you can't know which half to discard.

Q2. What is the maximum number of comparisons needed to find an element in a sorted array of 1024 elements using binary search?

- A. 10**
- B. 512
- C. 1024
- D. 1023

Answer: A. Binary search halves each iteration. $\log_2(1024) = 10$. So at most 10 comparisons are needed.

Q3. Which search is better for a small array ($n < 20$)?

- A. Binary search always
- B. Linear search** — overhead of binary search isn't worth it for small n
- C. Neither — use hash table
- D. Both are equal

Answer: B. For very small arrays, linear search can be faster because binary search has overhead (computing mid, branching). For $n < 20$, linear search's simplicity often wins.

Bubble, Selection, and Insertion Sort

Duration: 35 min · Difficulty: medium

Overview

Sorting arranges elements in ascending or descending order. Bubble sort repeatedly swaps adjacent elements. Selection sort finds the minimum and places it first. Insertion sort builds the sorted array one element at a time. All three are $O(n^2)$ in the worst case but differ in their best-case behavior and number of swaps.

Key Concepts

- Bubble Sort: repeatedly compare adjacent pairs, swap if out of order. Largest element 'bubbles up' to the end each pass.
- Selection Sort: find the minimum element in the unsorted portion, swap it with the first unsorted element.
- Insertion Sort: take each element and insert it into its correct position in the sorted portion (like sorting playing cards).
- All three have $O(n^2)$ worst-case time complexity
- Bubble sort and insertion sort are $O(n)$ best case (already sorted). Selection sort is always $O(n^2)$.
- Selection sort does the fewest swaps: $O(n)$. Bubble sort does $O(n^2)$ swaps.
- Insertion sort is stable (preserves order of equal elements). Selection sort is not.

Formulas

Σ Bubble sort passes: $n-1$. Comparisons in pass i : $n-i-1$. Total: $n(n-1)/2$

Σ Selection sort comparisons: $n(n-1)/2$. Swaps: $n-1$ (one per pass).

Σ Insertion sort comparisons: 1 (best) to i (worst) in pass i . Total worst: $n(n-1)/2$.

Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Stable?	Swaps (worst)
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	$O(n)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	—
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No	$O(n^2)$

Stable = preserves relative order of equal elements. Merge sort and quick sort are covered in Unit 5.

Bubble Sort Pass 1 (Largest Bubbles Up)

Initial: [5, 3, 8, 1, 2]

Compare 5,3 → swap: [3, 5, 8, 1, 2]

Compare 5,8 → ok: [3, 5, 8, 1, 2]

Compare 8,1 → swap: [3, 5, 1, 8, 2]

Compare 8,2 → swap: [3, 5, 1, 2, 8]

↑

8 is now in final position

Pass 2: [3, 5, 1, 2, 8] → [3, 1, 2, 5, 8] (5 in place)

Pass 3: [3, 1, 2, 5, 8] → [1, 2, 3, 5, 8] (3 in place)

Pass 4: [1, 2, 3, 5, 8] → [1, 2, 3, 5, 8] (sorted!)

Bubble Sort with Optimization

```
// Bubble Sort –  $O(n^2)$  worst,  $O(n)$  best (with optimization)
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0; // optimization: stop if no swaps

        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap adjacent elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }

        // If no swaps in this pass, array is sorted
        if (!swapped) break; //  $O(n)$  best case
    }
}
```

The 'swapped' flag is the key optimization. If a full pass happens without any swap, the array is already sorted and we can stop early. This makes the best case $O(n)$.

Selection Sort

```
// Selection Sort – always  $O(n^2)$ , but minimal swaps ( $O(n)$ )
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // Find the minimum in the unsorted portion [i..n-1]
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Swap the minimum with the first unsorted element
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }

        // Only 1 swap per pass – total n-1 swaps
    }
}
```

Selection sort finds the minimum and swaps it to the front. It always does $n-1$ swaps total (one per pass), which is the minimum possible for comparison-based sorting.

Insertion Sort (Like Sorting Cards)

```
// Insertion Sort –  $O(n^2)$  worst,  $O(n)$  best
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i]; // the element to insert
```

```

int j = i - 1;

// Shift elements > key one position right
while (j >= 0 && arr[j] > key) {
    arr[j + 1] = arr[j];
    j--;
}

// Insert key at its correct position
arr[j + 1] = key;
}
}

```

Think of sorting playing cards in your hand. Take each new card and insert it into its correct position among the already-sorted cards. Best case (already sorted) is $O(n)$ because the while loop never executes.

Common Mistakes

- ✗ Forgetting that bubble sort needs $n-1$ passes (not n)
- ✗ Inner loop in bubble sort: $j < n-i-1$, NOT $j < n-1$ (otherwise you re-compare sorted elements)
- ✗ Selection sort: comparing $arr[j]$ with $arr[i]$ instead of $arr[\text{minIndex}]$
- ✗ Insertion sort: starting i from 0 instead of 1 (element at index 0 is already 'sorted')
- ✗ Not knowing which sort is stable and which isn't

Exam Tips

- Know all three algorithms: bubble, selection, insertion — all $O(n^2)$ worst case
- Bubble sort best case is $O(n)$ with the swapped flag optimization
- Selection sort does the fewest swaps: $n-1$ total
- Insertion sort is efficient for nearly-sorted data — $O(n)$ best case
- Practice tracing each algorithm step by step with a small array

Practice Questions

Q1. Which sorting algorithm has the BEST best-case time complexity?

- A. Selection sort — $O(n^2)$
- B. Bubble sort — $O(n)$**
- C. Both are $O(n)$
- D. Neither — both are $O(n^2)$

Answer: B. Bubble sort (with the swapped optimization) is $O(n)$ in the best case (already sorted). Selection sort is always $O(n^2)$ even if the array is sorted, because it still scans for the minimum.

Q2. How many swaps does selection sort perform in the worst case?

- A. $O(n^2)$
- B. $O(n)$**
- C. $O(n \log n)$
- D. $O(1)$

Answer: B. Selection sort performs exactly $n-1$ swaps (one per pass). This is the minimum possible for comparison-based sorting, making it efficient when swaps are expensive.

Q3. Which sorting algorithm is most like sorting playing cards in your hand?

- A. Bubble sort
- B. Selection sort
- C. Insertion sort**
- D. Merge sort

Answer: C. Insertion sort works exactly like sorting cards: you pick up each new card and insert it into its correct position among the already-sorted cards.

Unit 5: Trees and Graphs

Weightage: 20%

Binary Trees and BST

Duration: 35 min · Difficulty: hard

Overview

A tree is a hierarchical data structure with nodes connected by edges. A binary tree has at most 2 children per node. A Binary Search Tree (BST) is a binary tree where left child < parent < right child. BSTs enable $O(\log n)$ search, insertion, and deletion when balanced.

Key Concepts

- Root: the top node. Parent: a node with children. Child: a node below a parent. Leaf: a node with no children.
- Depth of a node: number of edges from root to the node. Height of a tree: max depth.
- Binary Tree: each node has 0, 1, or 2 children.
- BST Property: left subtree values < node value < right subtree values
- Traversals: Inorder (Left-Root-Right), Preorder (Root-Left-Right), Postorder (Left-Right-Root)
- Inorder traversal of a BST gives elements in sorted order
- Balanced BST: height $\approx \log n \rightarrow O(\log n)$ operations
- Unbalanced BST (worst case): height = $n \rightarrow O(n)$ operations (degrades to linked list)

Formulas

Σ Max nodes at level i : 2^i

Σ Max nodes in tree of height h : $2^{(h+1)} - 1$

Σ Min height of n nodes: $\lceil \log_2(n) \rceil$

Σ BST search: compare key with node, go left if smaller, right if larger

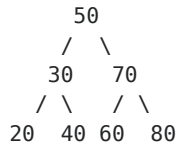
Tree Traversals

Traversal	Order	Use Case
Inorder	Left $\rightarrow$ Root $\rightarrow$ Right	BST $\rightarrow$ sorted output
Preorder	Root $\rightarrow$ Left $\rightarrow$ Right	Copy a tree, prefix expression
Postorder	Left $\rightarrow$ Right $\rightarrow$ Root	Delete a tree, postfix expression

Traversal	Order	Use Case
Level order	Level by level (BFS)	Find shortest path, level-wise processing

Inorder traversal of a BST produces elements in ascending order.

Binary Search Tree and Traversals



BST Property:

20 < 30 < 40 < 50 < 60 < 70 < 80

Inorder (Left-Root-Right):

20, 30, 40, 50, 60, 70, 80 ← SORTED!

Preorder (Root-Left-Right):

50, 30, 20, 40, 70, 60, 80

Postorder (Left-Right-Root):

20, 40, 30, 60, 80, 70, 50

BST Insertion and Inorder Traversal

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

// Create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert into BST – O(log n) average, O(n) worst
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value); // go left
    else if (value > root->data)
        root->right = insert(root->right, value); // go right
    // if equal, do nothing (no duplicates)

    return root;
}

// Inorder traversal – produces sorted output
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left); // visit left subtree
        printf("%d ", root->data); // visit root
        inorder(root->right); // visit right subtree
    }
}

```

```

}
}

int main() {
    struct Node* root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    // Output: 20 30 40 50 60 70 80
    inorder(root);
    return 0;
}

```

Insertion follows the BST property: smaller values go left, larger values go right. Inorder traversal visits left subtree first, then the node, then right subtree — which produces sorted output.

Common Mistakes

- ✗ Inserting duplicates — decide on a policy (ignore, or count them)
- ✗ Forgetting that BST operations are $O(n)$ in the worst case (unbalanced tree)
- ✗ Confusing inorder, preorder, and postorder traversal orders
- ✗ Not freeing tree nodes when deleting (memory leak)
- ✗ Forgetting the base case (`root == NULL`) in recursive traversals

Exam Tips

- Inorder traversal of a BST ALWAYS gives sorted output — remember this!
- Draw the tree before writing code
- Know the difference between depth (root to node) and height (node to deepest leaf)
- BST search is $O(\log n)$ when balanced, $O(n)$ when unbalanced
- Practice all three traversals with a small tree

Practice Questions

Q1. What is the inorder traversal of this BST? 50 / \ 30 70 / \ 20 40

- A. 50 30 20 40 70
- B. 20 30 40 50 70**
- C. 20 40 30 70 50
- D. 50 30 70 20 40

Answer: B. Inorder = Left-Root-Right. Start at 50: go left to 30, go left to 20. 20 has no left, so print 20. Back to 30, print 30. Go right to 40, print 40. Back to 50, print 50. Go right to 70, print 70. Result: 20 30 40 50 70.

Q2. What is the worst-case time complexity of searching in a BST?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

Answer: C. If the BST is completely unbalanced (all nodes on one side — like a linked list), the height is n , and search requires visiting all n nodes — $O(n)$.

Q3. Which traversal of a BST produces elements in ascending order?

- A. Preorder
- B. Inorder
- C. Postorder
- D. Level order

Answer: B. Inorder traversal (Left-Root-Right) visits smaller elements first (left subtree), then the node, then larger elements (right subtree). This produces ascending order.

Lernio AI · Learning OS 2.0 · CWIT R23

These notes are supplementary to your classroom learning. Always follow your teacher's instructions and the official CWIT syllabus.